

Plan de Pruebas — Módulo Bolsillos (FuBanking)

Asignatura	Validación y Verificación de Software
Proyecto	FuBanking — Banco digital
Módulo bajo prueba	Bolsillos (Pockets)
Fase	Pruebas unitarias (AAA + Mocks) y análisis estático (SonarQube)
Versión del plan	2.0 (actualiza el plan de la fase de caja negra/blanca)
Fecha	2026-09-14

1. Presentación del proyecto

FuBanking es un banco digital construido con arquitectura limpia (Clean Architecture) en cuatro capas:

Capa	Responsabilidad	Ejemplos en Bolsillos
Dominio	Entidades y reglas de negocio puras	Pocket, Account, Notification
Aplicación	Casos de uso (orquestración)	CreatePocket, UpdatePocket, ...
Infraestructura	Persistencia y servicios externos	SupabasePocketRepository
Presentación	Controladores, rutas, validadores HTTP	PocketController, pocket.validators

- Backend: Node.js + TypeScript + Express + Supabase.
- Frontend: Next.js + React.
- El dominio y la aplicación no conocen frameworks; dependen de interfaces de repositorio (Inversión de Dependencias), lo que permite inyectar dobles de prueba sin base de datos.

1.1 El módulo Bolsillos

Un bolsillo permite al usuario apartar dinero dentro de una cuenta para metas de ahorro, sin sacarlo de la cuenta. El saldo apartado se descuenta del saldo disponible de la cuenta. El módulo expone cinco funcionalidades (casos de uso):

1. Crear bolsillo — CreatePocket
2. Consultar bolsillos de la cuenta — GetAccountPockets
3. Actualizar bolsillo (nombre y/o monto) — UpdatePocket
4. Eliminar bolsillo — DeletePocket
5. Transferir saldo entre bolsillos — TransferPocketBalance

2. Objetivo del plan

Verificar, mediante pruebas unitarias automatizadas, que la lógica de negocio del módulo Bolsillos es correcta y robusta ante entradas válidas e inválidas, y evaluar la calidad interna del código mediante análisis estático (SonarQube/SonarCloud), cumpliendo los umbrales de un Quality Gate estricto.

3. Alcance

3.1 Elementos a probar (unidades)

Unidad	Tipo	Ubicación
Pocket	Entidad de dominio	domain/entities/Pocket.ts
CreatePocket	Caso de uso	use-cases/pocket/CreatePocket.ts
UpdatePocket	Caso de uso	use-cases/pocket/UpdatePocket.ts
DeletePocket	Caso de uso	use-cases/pocket/DeletePocket.ts
TransferPocketBalance	Caso de uso	use-cases/pocket/TransferPocketBalance.ts
GetAccountPockets	Caso de uso	use-cases/pocket/GetAccountPockets.ts

3.2 Fuera de alcance (para esta fase)

- Otros módulos (autenticación, créditos, tarjetas, perfil) — cubiertos por otros integrantes.
- Pruebas de integración por HTTP (controlador + Express + Supabase) — fase posterior.
- Pruebas de frontend — otro frente del equipo.
- El repositorio Supabase (SupabasePocketRepository) se cubre en la suite del equipo; aquí se sustituye por dobles de prueba para aislar la lógica de negocio.

4. Estrategia y enfoque de pruebas

4.1 Nivel: unitario y aislado

Cada caso de uso se prueba inyectándole dobles de los repositorios (cuenta, bolsillo, notificaciones) en vez de sus implementaciones reales de Supabase. Así la prueba ejercita solo la lógica del caso de uso, sin base de datos ni red.

4.2 Patrón AAA (Arrange – Act – Assert)

Toda prueba se estructura y rotula explícitamente en tres bloques:

- Arrange: preparar datos y dobles (ej. cuenta ACTIVA con saldo, bolsillo con monto).
- Act: invocar el método bajo prueba (useCase.execute(dto)).
- Assert: verificar el resultado o el error esperado (expect(...)).

4.3 Principios FIRST

Principio	Cómo se cumple
Fast	Solo memoria (dobles), sin BD ni red → milisegundos.
Independent	Cada prueba arma sus propios dobles (beforeEach/local); no comparten estado.
Repeatable	Fechas y datos fijos → mismo resultado en cualquier máquina.
Self-validating	Terminan en expect(...): pasan o fallan sin inspección manual.
Timely	Escritas junto al código de producción del módulo.

4.4 Los 5 tipos de dobles de prueba (Test Doubles, G. Meszaros)

Definidos y documentados en backend/src/tests/unit/pocket/test-doubles.ts.

#	Doble	Definición	Dónde se demuestra
1	Dummy	Se pasa solo para cumplir la firma; nunca se usa. Lanza si se invoca.	CreatePocket — monto negativo (prueba que no se notifica).
2	Fake	Implementación funcional pero simplificada (repos en memoria).	Repos base en casi todos los caminos felices.
3	Stub	Devuelve respuestas enlatadas fijas; ignora argumentos.	UpdatePocket y GetAccountPockets (cuenta enlatada).
4	Spy	Registra cómo fue llamado, para inspeccionarlo luego.	CreatePocket/DeletePocket (verifica la notificación).
5	Mock	Objeto pre-programado con expectativas verificables (vi.fn()).	CreatePocket (verifica updateBalance('acc-1', 380 000)).

4.5 Análisis estático — SonarQube / SonarCloud

Sobre el mismo módulo se ejecuta análisis estático para medir: Quality Gate, Bugs, Vulnerabilidades, Code Smells, Deuda Técnica, Duplicación e Instantánea, importando la cobertura generada por Vitest (lcov.info).

- Organización: alejodev11 · Project key: AlejoDev11_FuBanking · Host: sonarcloud.io.
- La cobertura de Bolsillos se incorpora al include de cobertura en backend/vitest.config.ts (Pocket.ts + use-cases/pocket/*.ts) y se fusiona en el lcov.info que consume Sonar.

5. Herramientas y entorno

Herramienta	Uso
Vitest 4	Framework de pruebas y aserciones
@vitest/coverage-v8	Cobertura (statements/branches/functions/lines)
vi.fn()	Construcción de Mocks y Spies
TypeScript / tsx	Lenguaje y ejecución
SonarCloud	Análisis estático y Quality Gate
Node.js	Runtime

Ejecución local (solo Bolsillos):

```
cd backend
npx vitest run src/tests/unit/pocket
```

Cobertura del módulo:

```
cd backend
npx vitest run src/tests/unit/pocket --coverage \
  --coverage.include='src/application/use-cases/pocket/**' \
  --coverage.include='src/domain/entities/Pocket.ts'
```

6. Diseño de casos y matriz de cobertura

Se combinan camino feliz y caminos de error (validaciones de dominio) de cada caso de uso. La suite consta de 58 pruebas distribuidas así:

Unidad	Pruebas	Ramas / condiciones principales cubiertas
CreatePocket	11	monto negativo, cuenta inexistente, ajena (FORBIDDEN), no operativa, saldo insuficiente, feliz, con/sin notificación
UpdatePocket	14	nombre vacío, monto negativo, bolsillo/cuenta inexistente, FORBIDDEN, no operativa, saldo insuficiente al ajustar, sin cambios, monto/nombre iguales
DeletePocket	8	bolsillo/cuenta inexistente, FORBIDDEN, no operativa, restitución de saldo, con/sin notificación
TransferPocketBalance	11	monto ≤ 0 , mismo bolsillo, origen/destino inexistente, distinta cuenta, cuenta inexistente/ajena/no operativa, saldo insuficiente, feliz
GetAccountPockets	4	cuenta inexistente, FORBIDDEN, lista con datos, lista vacía
Pocket (entidad)	10	monto negativo/NaN/no-numérico, monto 0, create() recorta nombre, updateName/updateAmount, toPublic
Total	58	

7. Criterios de entrada y de salida

Entrada: el código del módulo compila; dobles de prueba disponibles; entorno Vitest instalado.

Salida (definición de "hecho"):

- ☒ Las 58 pruebas pasan sin errores.
- ☒ Cobertura del módulo 100% (statements, branches, functions, lines).
- ☒ Se demuestran los 5 tipos de dobles con patrón AAA y principios FIRST.
- ☐ SonarCloud: Quality Gate en verde con calificación A (Bugs, Vulnerabilidades, Deuda).
- ☐ Quality Gate estricto: cobertura $\geq 90\%$, duplicación $\leq 2\%$, deuda técnica ≤ 90 min.

8. Métricas objetivo (alineadas a la rúbrica)

Métrica	Umbral objetivo	Estado del módulo
Cobertura de pruebas	$\geq 90\%$	100% ✓
Duplicación de código	$\leq 2\%$	por medir en Sonar
Deuda técnica	≤ 90 min	por medir en Sonar
Calificación fiabilidad/seguridad/mantenibilidad	A	por medir en Sonar
Complejidad ciclomática	evaluación manual + Sonar	UpdatePocket es el más complejo → candidato a refactor

9. Riesgos y dependencias

- Tabla pockets ausente en Supabase: el repositorio real degrada a un Map en memoria; por eso la integración queda para una fase posterior y esta fase usa dobles.
- Archivos compartidos (vitest.config.ts, sonar-project.properties): se editan de forma aditiva y coordinada con el equipo para evitar conflictos de merge.
- UpdatePocket concentra la mayor complejidad del módulo → riesgo de mantenibilidad; se abordará en la fase de refactorización con reejecución de SonarQube.

10. Responsables y entregables

Entregable	Ubicación
Suite de pruebas unitarias (7 archivos)	backend/src/tests/unit/pocket/
Definición de los 5 dobles	backend/src/tests/unit/pocket/test-doubles.ts
Este plan de pruebas	docs/plan-de-pruebas-bolsillos.md
Métricas manuales de Bolsillos	docs/Metricas-Bolsillos.xlsx (+ -Detalle.docx)
Resultados SonarCloud (instantáneas antes/después)	pendiente de corrida

11. Anexo — Mapa prueba → doble de prueba

Archivo de prueba	Dobles que demuestra
CreatePocket.test.ts	Dummy, Fake, Spy, Mock
UpdatePocket.test.ts	Fake, Stub
DeletePocket.test.ts	Fake, Spy
TransferPocketBalance.test.ts	Fake
GetAccountPockets.test.ts	Fake, Stub
Pocket.entity.test.ts	(unidad pura, sin dobles)